

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ**  
**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ**  
**«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ**  
**ІМЕНІ ІГОРЯ СІКОРСЬКОГО»**

**КАТЕДРА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ**

## **Алгоритми та методи обчислень**

### **Лабораторна робота №2**

**«Обчислювальна складність алгоритмів сортування»**

Виконав:  
студент групи ІО-32  
Крадожон М.  
Перевірів(-ла):  
Порєв В.

Київ – 2023

## Лабораторна робота №2

**Тема:** «Обчислювальна складність алгоритмів сортування»

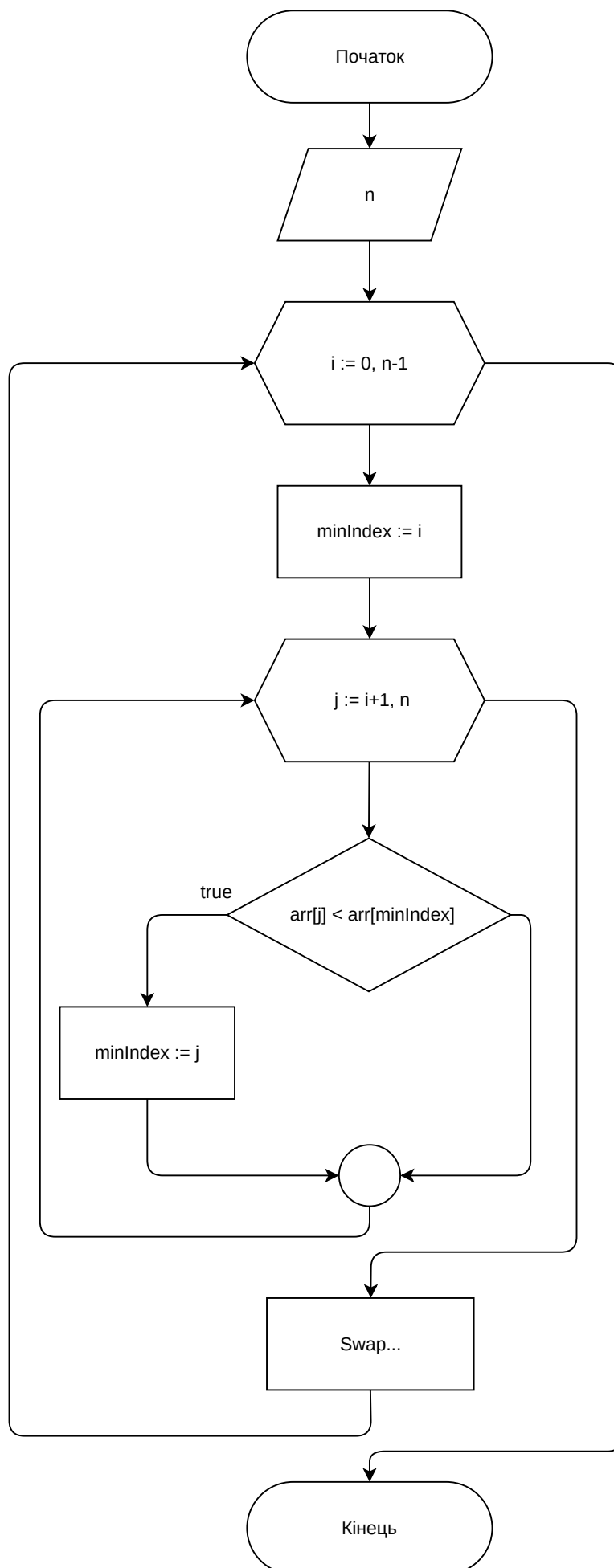
**Мета:** Закріплення навичок практичної оцінки алгоритмічної складності логічних алгоритмів на прикладі алгоритмів сортування.

**Завдання:** Використовуючи відповідний до варіанту алгоритм сортування написати програму сортування масиву даних. Застосовуючи дану програму, дослідити часову складність алгоритму сортування та порівняти її з теоретичною алгоритмічною складністю.

**Варіант:** 11

|    |                                                                  |          |
|----|------------------------------------------------------------------|----------|
| 11 | Алгоритм 3.2. Сортування вибором з пошуком мінімального елемента | $O(n^2)$ |
|----|------------------------------------------------------------------|----------|

## Блок-схеми алгоритмів:



## Код програми:

## Алгоритм

```
function selectionSort(arr: number[]): number[] {  
  const n = arr.length  
  
  for (let i = 0; i < n - 1; i++) {  
    let minIndex = i  
  
    for (let j = i + 1; j < n; j++) {  
      if (arr[j] < arr[minIndex]) {  
        minIndex = j  
      }  
    }  
  
    if (minIndex !== i) {  
      ;[arr[i], arr[minIndex]] = [arr[minIndex], arr[i]]  
    }  
  }  
  
  return arr  
}  
  
export { selectionSort }
```

# Програма та графічний інтерфейс

## App.tsx

```
import { ElementsProvider } from './common/elementsContext'
import Footer from './components/Footer'
import Main from './components/Main'

export default function App() {
  return (
    <ElementsProvider>
      <Main />
      <Footer />
    </ElementsProvider>
  )
}
```

## Main.tsx

```
import { Button } from '@ui/button'
import { Input } from '@ui/input'
import { Label } from '@ui/label'
import { Textarea } from '@ui/textarea'

import { useElementsContext } from '@common/elementsContext'
import generateArray from '@utils/generate-array'

export default function Main() {
  const {
    elements,
    setElements,
    initArray,
    setInitArray,
    resultArray,
    executionTime
  } = useElementsContext()

  const handleGenerate = () => {
    const numElements = generateArray(parseInt(elements, 10))
    setInitArray(numElements.join(', '))
  }

  return (
    <main className="flex h-screen flex-col gap-5 p-5">
      <h1 className="mb-2 text-2xl font-bold">Сортування вибору</h1>
      <div className="grid grid-cols-2 items-end justify-items-center">
        <div className="grid w-full max-w-sm items-center gap-1.5">
          <Label htmlFor="elements">Кількість елементів</Label>
          <div className="flex gap-2">
            <Input
              type="number"
              id="elements"
              placeholder="Число"
              onChange={(e) => setElements(e.target.value)}
            />
          </div>
        </div>
      </div>
    </main>
  )
}
```

```

    />
    <Button variant="outline" onClick={handleGenerate}>
      Згенерувати
    </Button>
  </div>
</div>
{executionTime !== null && (
  <p className="text-sm text-gray-500">
    Час виконання: {executionTime.toFixed(10)} мс
  </p>
)}
</div>

<div className="flex h-full max-h-96 gap-x-2">
  <div className="grid w-full gap-1.5">
    <Label htmlFor="init-array">Початковий масив</Label>
    <Textarea
      className="h-54 resize-none"
      placeholder="Числа через кому"
      id="init-array"
      value={initArray}
      onChange={(e) => setInitArray(e.target.value)}
    />
  </div>

  <div className="grid w-full gap-1.5">
    <Label htmlFor="result">Відсортований масив</Label>
    <Textarea
      className="h-54 resize-none"
      id="result"
      value={resultArray}
      readOnly
    />
  </div>
</div>
</main>
)
}

```

## Footer.tsx

```

import { Button } from '@ui/button'
import { Info } from 'lucide-react'

import showDialog from '@/utils/show-dialog'

import Calculate from './Calculate'
import CreatePlot from './CreatePlot'
import ImportFile from './ImportFile'

export default function Footer() {
  const handleInfo = () => {
    showDialog(

```

```

    'info',
    'Лабораторна робота',
    'Максим Крадожон ІО-32\nВаріант 11'
  )
}

return (
  <footer className="bottom-0 flex h-15 w-full items-center justify-between border-t border-neutral-300 px-3">
    <Button variant="ghost" size="icon" onClick={handleInfo}>
      <Info size={40} />
    </Button>

    <div className="flex gap-3">
      <ImportFile />
      <CreatePlot />
      <Calculate />
    </div>
  </footer>
)
}

```

## Calculate.tsx

```

import { useElementsContext } from '@common/elementsContext'
import { Button } from '@components/ui/button'
import { selectionSort } from '@lib/algorithm'

export default function Calculate() {
  const { initArray, setResultArray, setExecutionTime } = useElementsContext()

  const handleGenerate = () => {
    const array = initArray.split(',').map(Number)
    const startTime = performance.now()
    const result = selectionSort(array)
    const endTime = performance.now()
    setResultArray(result.join(', '))
    setExecutionTime(endTime - startTime)
  }

  return (
    <Button variant="default" onClick={handleGenerate}>
      Обчислити
    </Button>
  )
}

```

## CreatePlot.tsx

```

import { useRef, useState } from 'react'
import { Line } from 'react-chartjs-2'
import { Chart, registerables } from 'chart.js'

import { Button } from '@components/ui/button'

```

```

import {
  Dialog,
  DialogContent,
  DialogDescription,
  DialogFooter,
  DialogTitle,
  DialogTrigger
} from '@components/ui/dialog'
import { ScrollArea } from '@components/ui/scroll-area'
import generateArray from '@utils/generate-array'
import { measureStats } from '@utils/measure-stats'

Chart.register(...registerables)

export default function CreatePlot() {
  const chartTimeRef = useRef<Chart<'line'> | null>(null)
  const [data, setData] = useState<number[][]>([])
  const chartOpsRef = useRef<Chart<'line'> | null>(null)
  const [operationsData, setOperationsData] = useState<number[][]>([])

  const handleGeneratePlot = () => {
    const sizes = [10, 50, 100, 500, 1000, 5000]

    const results = sizes.map((size) => {
      const array = generateArray(size)
      return measureStats(array)
    })

    setData(sizes.map((size, i) => [size, results[i].time]))
    setOperationsData(sizes.map((size, i) => [size, results[i].operations]))
  }

  const handleExport = () => {
    if (!chartTimeRef.current || !chartOpsRef.current) return

    const canvas1 = chartTimeRef.current.canvas.toDataURL('image/png')
    const canvas2 = chartOpsRef.current.canvas.toDataURL('image/png')

    window.api!.saveImage(canvas1)
    window.api!.saveImage(canvas2)
  }

  return (
    <Dialog>
      <DialogTrigger>
        <Button variant="outline" onClick={handleGeneratePlot}>
          Побудувати графік
        </Button>
      </DialogTrigger>

      <DialogContent className="h-auto w-auto">
        <DialogTitle>Графіки виконання алгоритму</DialogTitle>

```



```

<ScrollArea className="h-56 w-115">
  <DialogDescription className="h-56 w-115">
    {data.length > 0 && (
      <>
        <Line
          ref={chartTimeRef}
          data={{
            labels: data.map(([size]) => size),
            datasets: [
              {
                label: 'Час виконання (розмір масиву/мс)',
                data: data.map([, time]) => time),
                borderColor: 'blue',
                borderWidth: 2
              }
            ]
          }}
          options={{ responsive: true, maintainAspectRatio: false }}
        />

        <Line
          ref={chartOpsRef}
          data={{
            labels: operationsData.map(([size]) => size),
            datasets: [
              {
                label: 'Кількість операцій (розмір масиву/операції)',
                data: operationsData.map([, ops]) => ops),
                borderColor: 'red',
                borderWidth: 2
              }
            ]
          }}
          options={{ responsive: true, maintainAspectRatio: false }}
        />
      </>
    )}
  </DialogDescription>
</ScrollArea>

<DialogFooter className="items-end">
  <Button variant="outline" onClick={handleExport}>
    Експортувати графіки
  </Button>
</DialogFooter>
</DialogContent>
</Dialog>
)
}

```

## ImportFile.tsx

```
import { useEffect } from 'react'
```

```

import { Button } from '@ui/button'
import { useFilePicker } from 'use-file-picker'

import { useElementsContext } from '@common/elementsContext'
import showDialog from '@utils/show-dialog'

export default function ImportFile() {
  const { openFilePicker, filesContent } = useFilePicker({
    accept: '.txt',
    multiple: false,
    readAs: 'Text'
  })
  const { setInitArray } = useElementsContext()

  useEffect(() => {
    if (!filesContent.length) return
    const content = filesContent[0].content

    const isValid = content
      .split(',')
      .every((item) => !isNaN(Number(item.trim()))))
    if (!isValid) {
      showDialog('error', 'Помилка', 'Файл містить некоректні дані')
      return
    }

    setInitArray(content)
  }, [filesContent, setInitArray])

  return (
    <Button variant="ghost" onClick={() => openFilePicker()}>
      Вставити з файлу
    </Button>
  )
}

```

# Context

## elementsContext.tsx

```
import {
  createContext,
  Dispatch,
  ReactNode,
  SetStateAction,
  useContext,
  useState
} from 'react'

interface ElementsContextType {
  elements: string
  setElements: Dispatch<SetStateAction<string>>
  initArray: string
  setInitArray: Dispatch<SetStateAction<string>>
  resultArray: string
  setResultArray: Dispatch<SetStateAction<string>>
  executionTime: number | null
  setExecutionTime: Dispatch<SetStateAction<number | null>>
}

export const ElementsContext = createContext<ElementsContextType | undefined>(
  undefined
)

export const ElementsProvider = ({ children }: { children: ReactNode }) => {
  const [elements, setElements] = useState('')
  const [initArray, setInitArray] = useState('')
  const [resultArray, setResultArray] = useState('')
  const [executionTime, setExecutionTime] = useState<number | null>(null)

  return (
    <ElementsContext.Provider
      value={{
        elements,
        setElements,
        initArray,
        setInitArray,
        resultArray,
        setResultArray,
        executionTime,
        setExecutionTime
      }}
    >
      {children}
    </ElementsContext.Provider>
  )
}

export const useElementsContext = () => {
```

```

const context = useContext(ElementsContext)

if (!context) {
  throw new Error('ElementsContext must be used within an ElementsProvider')
}

return context
}

```

## Utils

### generate-array.tsx

```

function generateArray(numElements: number): number[] {
  if (!isNaN(numElements) && numElements > 0) {
    const array = Array.from({ length: numElements }, () =>
      Math.floor(Math.random() * 100)
    )
    return array
  }
  return []
}

export default generateArray

```

### measure-stats.tsx

```

import { selectionSortWithStats } from '@/lib/selection-sort-stars'

function measureStats(arr: number[]) {
  const copy = [...arr]

  const startTime = performance.now()
  const { comparisons, swaps } = selectionSortWithStats(copy)
  const time = performance.now() - startTime

  return { time, operations: comparisons + swaps }
}

export { measureStats }

```

### show-dialog.tsx

```

const showDialog = (type: 'info' | 'error', title: string, message: string) => {
  window.api.showDialog({ type, title, message })
}

export default showDialog

contextBridge.exposeInMainWorld('api', {
  showDialog(details: Electron.MessageBoxOptions) {
    ipcRenderer.invoke('show-dialog', details)
  }
})

```

```
ipcMain.handle('show-dialog', async (event, details) => {  
  dialog.showMessageBox({  
    type: details.type,  
    buttons: ['OK'],  
    title: details.title,  
    detail: details.detail,  
    message: details.message  
  })  
})
```



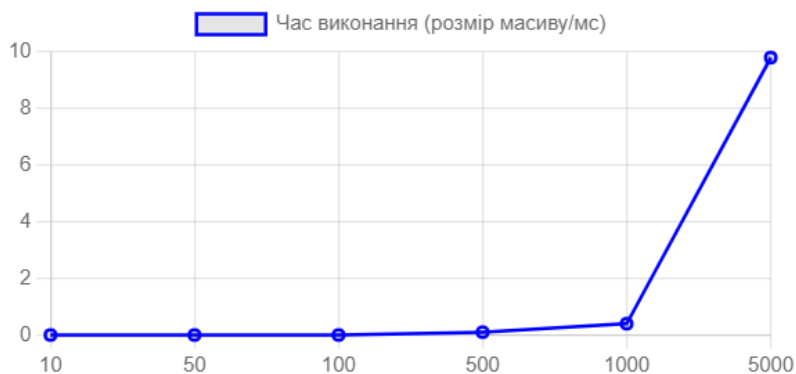
## Сортування вибору

Кількість елемент

## Початковий маси

77, 33, 91, 42, 1, 9  
41, 99, 99, 42, 27,  
72, 23, 77, 63, 34,  
59, 47, 32, 46, 19,  
95, 55, 85, 95, 24,  
25, 49, 85, 47, 6, 8  
86, 27, 57, 11, 18,  
43, 59, 72, 52, 99,  
29, 8, 42, 74, 42, 9  
18, 15, 49, 9, 82, 3

## Графіки виконання алгоритму



Експортувати графіки



Вставити з файлу

### Побудувати графік

## Сортування вибору

Кількість елемент

## Початковий маси

77, 33, 91, 42, 1, 9  
41, 99, 99, 42, 27,  
72, 23, 77, 63, 34,  
59, 47, 32, 46, 19,  
95, 55, 85, 95, 24,  
25, 49, 85, 47, 6, 8  
86, 27, 57, 11, 18,  
43, 59, 72, 52, 99,  
29, 8, 42, 74, 42, 9  
18, 15, 49, 9, 82, 3

## Графіки виконання алгоритму



Експортувати графіки

①

Вставити з файлу

Побудувати графік

### **Аналіз результатів:**

Ця програма реалізує алгоритм сортування вибору з пошуком мінімального елемента. Вона дозволяє користувачу ввести розмір масиву, згенерувати масив випадкових чисел або завантажити масив з файлу, після чого відобразити невідсортований масив.

Кнопка "Обчислити" запускає процес сортування, після чого відсортований масив відображається в окремому полі, а також виводиться час, необхідний для сортування. Кнопка "Побудувати графік" дозволяє побудувати графіки залежності часу виконання алгоритму від розміру масиву даних і теоретичної обчислювальної складності.

Для вимірювання часу сортування використовується модуль `node:perf_hooks`. Програма дозволяє завантажити масив з файлу та вивести відсортований масив, а також провести порівняльний аналіз залежності часу виконання від розміру масиву з теоретичною обчислювальною складністю алгоритму.

**Висновок:** Під час виконання лабораторної роботи, я написав програму на мові програмування JavaScript для обчислення алгоритму сортування вибору з пошуком мінімального елемента. Створив блок-схеми для кожного з алгоритмів. Забезпечив введення даних з клавіатури або документу та обробку всіх можливих помилок.